INFORME DE LABORATORIO

INFORMACIÓN BÁSICA

ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Pantallas múltiples con NavigationBar y comunicación entre Composables en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	05	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	B
FECHA DE PRESENTACIÓN:	03/10/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Hancco Mullisaca Sergio Danilo - Huacani Jara Denise Andrea				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Link (Propuesta 1): https://github.com/shanccom/IDNP_Repo/tree/main/lab_05_CampusNavigator_Compose/CampusNavigator_Composes

Link (Propuesta 2): https://github.com/DeniseHuacani/IDNP-LABORATORIO/tree/main/lab05/CampusNavigator_Compose

EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Enunciado: Construir una aplicación con tres pantallas (Home, Edificios y Mapa) navegables desde una barra inferior de navegación, comunicando la selección de un edificio mediante una función lambda.

Desarrollo

Paso 1. Creación del proyecto y definición de la `sealed class Screen`

Se creó el proyecto en Android Studio y se definió la clase sellada `Screen` con sus tres objetos, cada uno con su propiedad `route` (String) y `label` (String):

Además se agregaron las importaciones que se utilizan en los siguientes pasos:



Figura 1: Paso 1. Configuración de importaciones y definición de la sealed class Screen.

Paso 2. Dependencias necesarias en `build.gradle.kts`

Se verificó el archivo `build.gradle.kts` (Module `:app`) y se confirmaron las dependencias requeridas dentro del bloque `dependencies { ... }`:

La librería `androidx.navigation:navigation-compose` aporta el `NavHost` y el `NavController`, mientras que `androidx.compose.material:material-icons-extended` provee los iconos `Icons.Default.Home`, `Icons.Default.List` e `Icons.Default.Place` de la barra de navegación. Una vez agregadas, se ejecutó la sincronización del proyecto con el botón **Sync Now** antes de continuar.

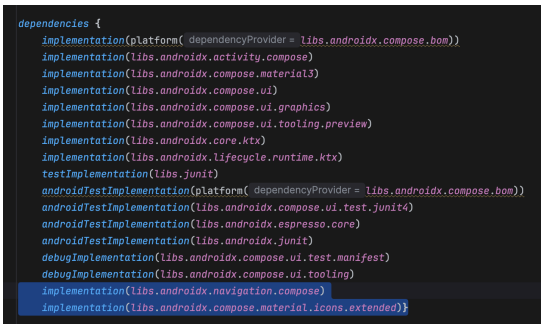


Figura 2: Paso 2. Dependencias añadidas en build.gradle.kts y sincronización del proyecto.

Paso 3. Composable raíz con `NavController` y estado del edificio

En el Composable raíz se creó el `navController` y la variable de estado que almacenará el último edificio consultado:

`rememberNavController()` crea el controlador de navegación persistente entre recomposiciones, mientras que `mutableStateOf("Ninguno")` inicializa el estado con el valor por defecto **«Ninguno»** y permite actualizarlo mediante la lambda.



Figura 3: Paso 3. Declaración de NavController y variable de estado edificioSeleccionado.

Paso 4. `Scaffold` con la `NavigationBar` inferior

Se envolvió la interfaz dentro de un `Scaffold(bottomBar = { ... })` para estructurar la pantalla. En el `bottomBar` se declaró una `NavigationBar` con un `NavigationBarItem` por cada `Screen`, marcando como seleccionado el que coincide con la ruta actual obtenida mediante `currentBackStackEntryAsState()`, y llamando a `navController.navigate(screen.route)` al pulsarlo:

De esta forma la barra inferior controla qué ítem se muestra resaltado según la ruta activa de la pila de navegación.

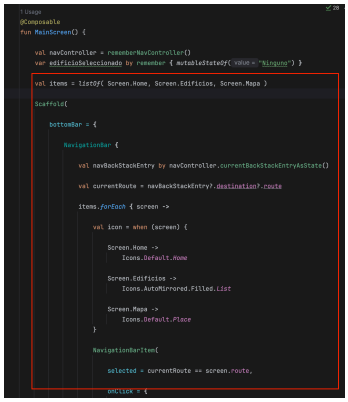


Figura 4: Paso 4. Scaffold con NavigationBar y NavigationBarItems.

Paso 5. `NavHost` con las tres rutas

Dentro del `padding` entregado por el `Scaffold` se declaró el `NavHost` con `startDestination = Screen.Home.route` y un bloque `composable(route) { ... }` para cada pantalla:

La lambda pasada a `EdificiosScreen` es la que comunica la selección hacia el Composable raíz.



Figura 5: Paso 5. NavHost con las rutas home, edificios y mapa.

Paso 6. Implementación de `HomeScreen`

Se implementó `HomeScreen(edificioSeleccionado: String)` mostrando un texto de bienvenida y otro con el valor recibido:

Este Composable es completamente stateless: recibe el valor como parámetro y Compose lo redibuja automáticamente cuando el estado del Composable raíz cambia.



```
1 Usage
@Composable
fun HomeScreen(
    edificioSeleccionado: String
) {

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding( all = 16.dp),

        horizontalAlignment = Alignment.CenterHorizontally
    ) {

        Text(
            text = "Bienvenido a Campus Navigator"
        )

        Text(
            text = "Último edificio consultado:"
        )

        Text(
            text = edificioSeleccionado
        )
    }
}
```

Figura 6: Paso 6. Pantalla Home con el último edificio consultado.

Paso 7. Implementación de `EdificiosScreen`

Se implementó `EdificiosScreen(onEdificioSeleccionado: (String) -> Unit)` con una `LazyColumn` que lista al menos 4 edificaciones (nombre + botón). Al pulsar un ítem se invoca `onEdificioSeleccionado(nombre)`:

El tipo `(String) -> Unit` define la función lambda de retorno, que desacopla la lista de edificios del estado global de la navegación.



```
1 Usage
@Composable
fun EdificiosScreen(
    onEdificioSeleccionado: (String) -> Unit
) {

    val edificios = listOf(
        "Biblioteca Central",
        "Pabellón A",
        "Pabellón B",
        "Auditorio"
    )

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding( all = 16.dp)
    ) {

        items(edificios) { nombre ->

            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 8.dp),

                verticalAlignment = Alignment.CenterVertically
            ) {

                Text(
```

Figura 7: Paso 7. Lista de edificios en LazyColumn con botón Ver.

Paso 8. Implementación de `MapaScreen`

Se implementó `MapaScreen()` con un `Text` marcador de posición, dejando el mapa real para una posterior integración:

La lambda que actualiza `edificioSeleccionado` ya se declaró en el Paso 5, al definir `composable(Screen.Edificios.route)` dentro del `NavHost`; por ello no se requiere ningún parámetro adicional aquí.

```
1 Usage
@Composable
fun MapaScreen() {

    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {

        Text(
            text = "Mapa de ubicaciones (pendiente de integrar)"
        )
    }
}
```

Figura 8: Paso 8. Pantalla Mapa con marcador de posición.

Paso 9. `MainActivity` invocando a `MainScreen()`

En `MainActivity`, dentro de `onCreate()`, se reemplazó el contenido de `setContent { }` (el que trae la plantilla por defecto, con `Scaffold` y `Greeting`) para invocar a `MainScreen()`:

De esta forma, el tema de la plantilla se conserva y la navegación queda centralizada en el Composable raíz `MainScreen()`.

```
1 Usage
class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            CampusNavigator_ComposeTheme {
                MainScreen()
            }
        }
    }
}
```

Figura 9: Paso 9. MainActivity llamando a MainScreen().

Paso 10. Ejecución y verificación en el emulador

Se ejecutó la aplicación en el emulador y se verificó que, al pulsar cada ítem de la barra inferior, se muestra la pantalla correspondiente conservando el resaltado del ítem activo. Luego, al seleccionar un edificio en la pestaña **Edificios** y volver a **Home**, se confirmó que `HomeScreen` muestra el valor actualizado, lo que demuestra el correcto funcionamiento de la comunicación mediante lambda entre pantallas.

Pruebas de Integrante 1

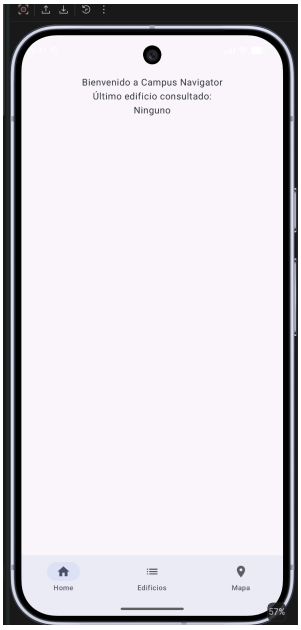


Figura 10: (a) Pantalla Home inicial («Ninguno»)

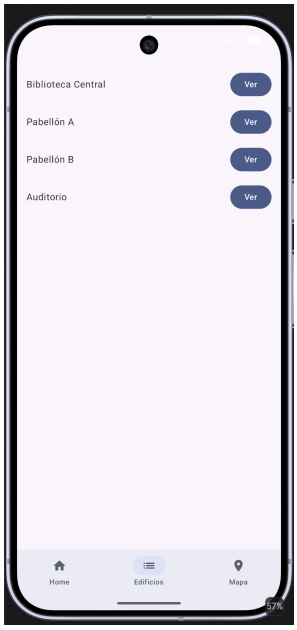


Figura 11: (b) Pantalla Edificios (Lista)

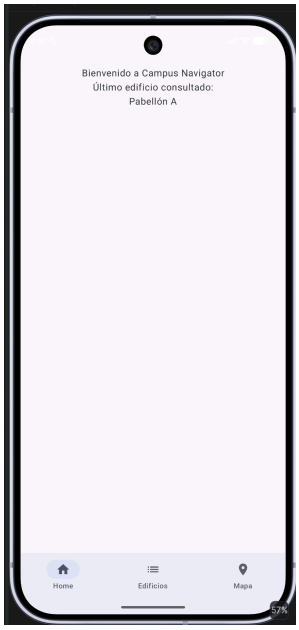


Figura 12: (c) Home tras seleccionar Pabellón A



Figura 13: (d) Pantalla Mapa (Marcador)

Pruebas de Integrante 2

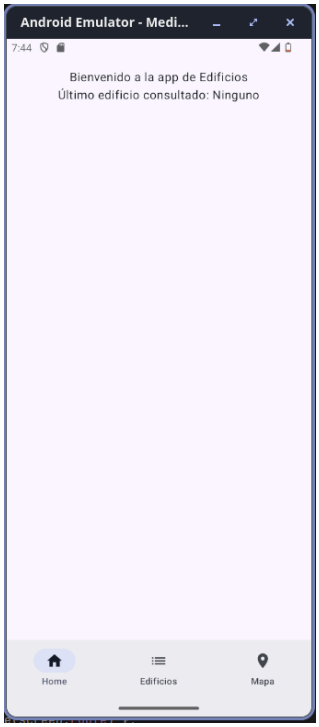


Figura 14: (a) Pantalla Home inicial («Ninguno»)

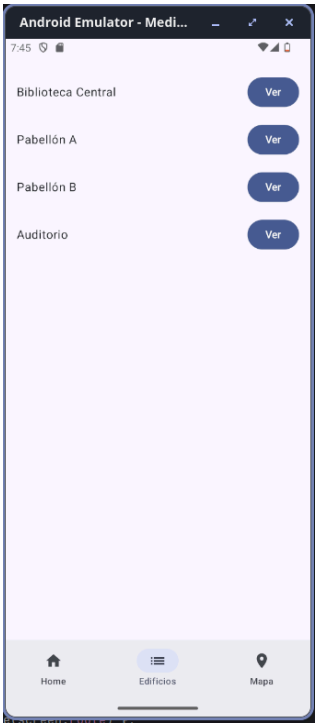


Figura 15: (b) Pantalla Edificios (Lista)

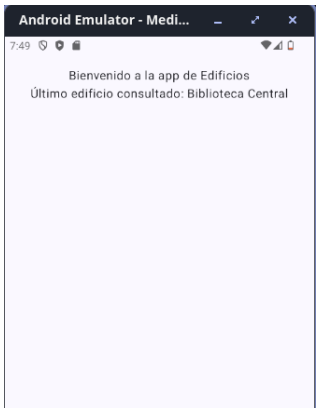


Figura 16: (c) Home tras seleccionar «Blblioteca central»

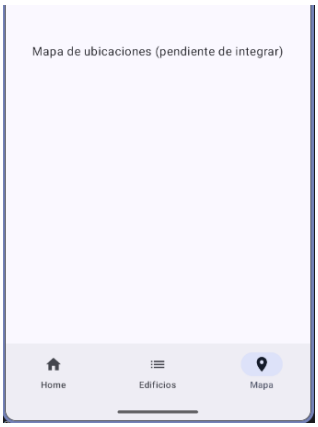


Figura 17: (d) Pantalla Mapa (Marcador)

EJERCICIOS/PROBLEMAS PROPUESTOS

Enunciado

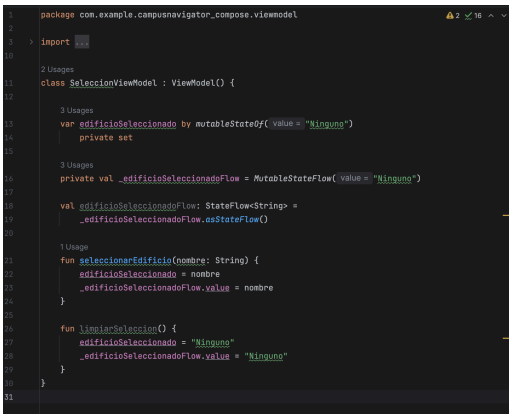
Tomando como referencia el ejercicio resuelto por el docente, reemplaza la comunicación por lambda entre Edificios y Home por un ViewModel compartido, y agrega una cuarta pestaña:

1. Crea una clase `SeleccionViewModel : ViewModel()` con una propiedad `mutableStateOf<String>` (o `StateFlow`) para almacenar el edificio seleccionado.
2. Obtén una única instancia de `SeleccionViewModel` con `viewModel()` en el Composable raíz y pásala a `EdificiosScreen` y a `HomeScreen` (en lugar de la lambda del ejercicio resuelto).
3. En `EdificiosScreen`, al seleccionar un edificio, actualiza directamente el valor en el ViewModel en vez de invocar una función lambda.
4. En `HomeScreen`, lee el valor del ViewModel (con `collectAsState()` si usaste `StateFlow`) y muéstralo, verificando que el comportamiento visual sea el mismo que con la lambda.
5. Agrega una cuarta pestaña «Perfil» (u otra que prefieras) con un Composable simple de marcador de posición, y su `NavigationBarItem` correspondiente.

Propuesta 1

Paso 1 : Crea una clase SeleccionViewModel : ViewModel() con una propiedad mutableStateOf (o StateFlow) para almacenar el edificio seleccionado.

Se creó el archivo `viewmodel/SeleccionViewModel.kt` con la clase `SeleccionViewModel : ViewModel()`, que expone el edificio seleccionado tanto con `mutableStateOf` (con `private set`) como con un `StateFlow` construido a partir de un `MutableStateFlow`, además de los métodos `seleccionarEdificio()` y `limpiarSeleccion()`:



```
package com.example.campusnavigator_compose.viewmodel

import ...

2 Usages
class SeleccionViewModel : ViewModel() {

    3 Usages
    var edificioSeleccionado by mutableStateOf( value = "Ninguno")
        private set

    3 Usages
    private val _edificioSeleccionadoFlow = MutableStateFlow( value = "Ninguno")

    val edificioSeleccionadoFlow: StateFlow<String> =
        _edificioSeleccionadoFlow.asStateFlow()

    1 Usage
    fun seleccionarEdificio(nombre: String) {
        edificioSeleccionado = nombre
        _edificioSeleccionadoFlow.value = nombre
    }

    fun limpiarSeleccion() {
        edificioSeleccionado = "Ninguno"
        _edificioSeleccionadoFlow.value = "Ninguno"
    }
}
```

Figura 18: Clase SeleccionViewModel con el estado en `mutableStateOf` y en `StateFlow`.

En `MainActivity.kt` se eliminó el `var edificioSeleccionado by remember {mutableStateOf("Ninguno") }` que usaba la solución con lambda (y con él los imports `mutableStateOf` y `setValue`), y en `libs.versions.toml` más `app/build.gradle.kts` se agregaron `lifecycle-viewmodel-ktx` y `lifecycle-viewmodel-compose` en la versión 2.11.0, porque el proyecto todavía no tenía ninguna dependencia de ViewModel. El valor por defecto quedó además expuesto como la constante `SeleccionViewModel.SIN_SELECCION`, que es la que usa `HomeScreen` para distinguir si todavía no hay ningún edificio consultado.

Paso 2 : Obtén una única instancia de SeleccionViewModel con viewModel() en el Composable raíz y pásala a EdificiosScreen y a HomeScreen (en lugar de la lambda del ejercicio resuelto).

En el Composable raíz `MainScreen` se obtiene una única instancia con `viewModel()` y esa misma instancia se entrega a las dos pantallas dentro del `NavHost`, en lugar de la lambda del ejercicio resuelto. Como `viewModel()` conserva la instancia por `ViewModelStoreOwner` (el Activity), ambas pantallas reciben exactamente el mismo objeto aunque se abran y cierren varias veces:

```
@Composable
fun MainScreen() {

```

```
    val navController = rememberNavController()

    val seleccionViewModel: SeleccionViewModel = viewModel()

    val items = listOf( Screen.Home, Screen.Edificios, Screen.Mapa, Screen.Perfil )

    Scaffold(
        bottomBar = { /* NavigationBar con un NavigationBarItem por screen */ }
    ) { paddingValues ->

        NavHost(
            navController = navController,
            startDestination = Screen.Home.route,
            modifier = Modifier.padding(paddingValues)
        ) {

            composable(Screen.Home.route) {

                HomeScreen(
                    viewModel = seleccionViewModel
                )
            }

            composable(Screen.Edificios.route) {

                EdificiosScreen(
                    viewModel = seleccionViewModel,
                    navController = navController
                )
            }

            // composable(Screen.Mapa.route) { MapaScreen() }
            // composable(Screen.Perfil.route) { PerfilScreen() }
        }
    }
}
```

El nuevo import `androidx.lifecycle.viewmodel.compose.viewModel` es el único que se requirió en este punto. El resto del ajuste se detalla en el punto 3, donde se escriben las nuevas firmas de los Composables.

Paso 3 : En EdificiosScreen, al seleccionar un edificio, actualiza directamente el valor en el ViewModel en vez de invocar una función lambda.

Recibida la instancia compartida del punto 2, `EdificiosScreen` cambió su firma de `onEdificioSeleccionado: (String) -> Unit` a `viewModel: SeleccionViewModel` más `navController: NavHostController`, y el botón **Ver** actualiza el ViewModel antes de navegar en lugar de invocar la lambda:

```
@Composable
fun EdificiosScreen(
    viewModel: SeleccionViewModel,
    navController: NavHostController
) {
```

```
    // ...
    items(edificios) { edificio ->
        // ...
        FilledTonalButton(
            onClick = {

                viewModel.seleccionarEdificio(edificio.nombre)

                navController.navigate(Screen.Home.route) {
                    popUpTo(Screen.Home.route) {
                        inclusive = false
                    }
                    launchSingleTop = true
                }
            }
        ) {

            Text("Ver")
        }
    }
}
```

La lambda ya no existe en ningún lado: la pantalla conversa directamente con el ViewModel. Se llamó al método `seleccionarEdificio()` en vez de asignar la propiedad porque el `private set` impide escribirla desde la pantalla y una asignación directa dejaría el `StateFlow` desincronizado. Como único import nuevo se agregó `androidx.navigation.NavHostController`, necesario para el tipo del parámetro.

Paso 4 : En HomeScreen, lee el valor del ViewModel (con collectAsState() si usaste StateFlow) y muéstralo, verificando que el comportamiento visual sea el mismo que con la lambda.

En `HomeScreen` se dejó de leer la propiedad del ViewModel directamente y se declaró una variable local que **observa** el `StateFlow` mediante `collectAsState()`, que es la que muestra la tarjeta de **«Último edificio consultado»**. Solo se agregó el import de `collectAsState` (`getValue` ya estaba) y el archivo `SeleccionViewModel.kt` no se modificó:

```
@Composable
fun HomeScreen(
    viewModel: SeleccionViewModel
) {

    val edificioSeleccionado by viewModel.edificioSeleccionadoFlow.collectAsState()
    val sinSeleccion = edificioSeleccionado == SeleccionViewModel.SIN_SELECCION
    // ...
    Card(
        // ...
    ) {

        Text(
            text = "Último edificio consultado",
            style = MaterialTheme.typography.labelMedium,

            color = MaterialTheme.colorScheme.onSecondaryContainer
        )
```

```
19
20          Text(
21              text = edificioSeleccionado,
22              style = MaterialTheme.typography.titleMedium,
23
24              color = MaterialTheme.colorScheme.onSecondaryContainer
25          )
26      }
27  }
```

El valor leído además permite alternar el ícono de la tarjeta entre búsqueda y ubicación mediante `sinSeleccion`, es decir, la suscripción al flujo también alimenta elementos de la interfaz distintos del texto.

Paso 5 : Agrega una cuarta pestaña «Perfil» (u otra que prefieras) con un Composable simple de marcador de posición, y su NavigationBarItem correspondiente.

Se creó el objeto `Screen.Perfil` en la `sealed class` con su `route` y su `label`, se registró su `NavigationBarItem` en la barra inferior y se añadió `composable(Screen.Perfil.route)` en el `NavHost` para invocar a `PerfilScreen`, un Composable simple de marcador de posición. La navegación inferior queda con cuatro destinos y el resaltado del ítem activo se mantiene porque sigue dependiendo de la ruta actual de la pila de navegación.

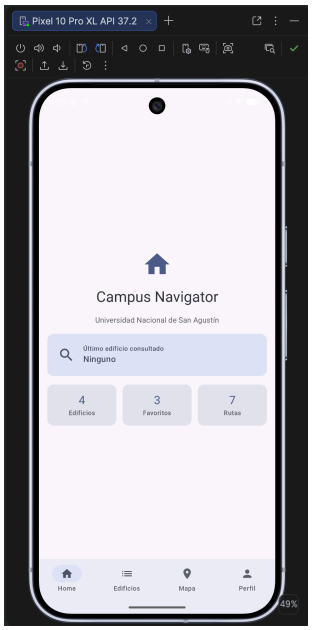


Figura 19: (a) Pestaña Home («Ninguno»)

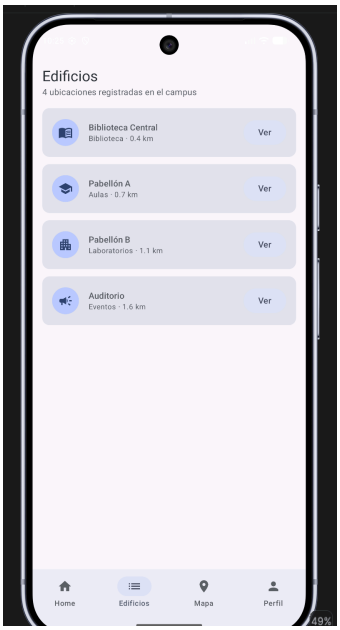


Figura 20: (b) Pestaña Edificios

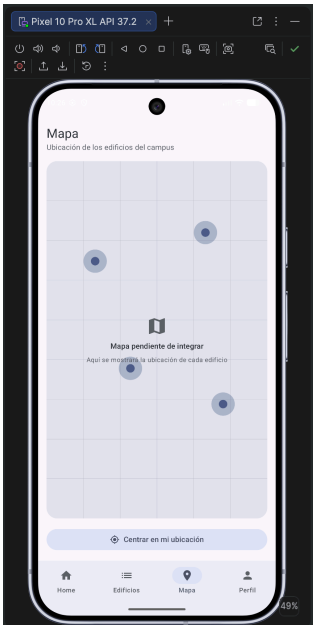


Figura 21: (c) Pestaña Mapa

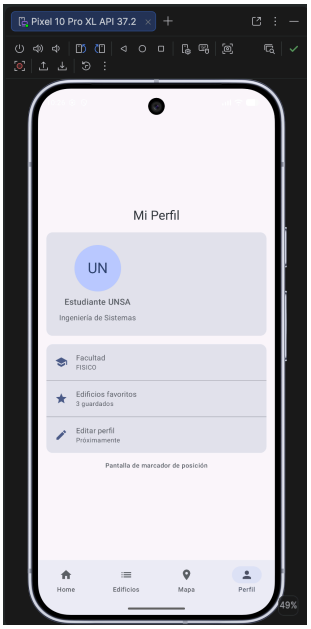


Figura 22: (d) Cuarta pestaña Perfil

Propuesta 2

Paso 1: Crear la clase `SeleccionViewModel`

Enunciado: Crea una clase `SeleccionViewModel : ViewModel()` con una propiedad `mutableStateOf<String>` (o `StateFlow`) para almacenar el edificio seleccionado.

Implementación: Se reemplazó la clase vacía `SelectViewModel` por `SeleccionViewModel`, que hereda de `ViewModel`. Se usó `mutableStateOf<String>("Ninguno")` para guardar el edificio seleccionado, con el valor inicial «Ninguno». Al ser un estado de Compose, cualquier Composable que lea esta propiedad se recompone automáticamente cuando cambia. La propiedad tiene `private set` para que solo pueda modificarse mediante el método `seleccionarEdificio()`. En este paso la app sigue funcionando con la lambda, porque el ViewModel todavía no se usa.

Código agregado:

```
class SeleccionViewModel : ViewModel() {
    var edificioSeleccionado by mutableStateOf<String>("Ninguno")
        private set

    fun seleccionarEdificio(nombre: String) {
        edificioSeleccionado = nombre
    }
}
```

Paso 2: Obtener una única instancia con `viewModel()`

Enunciado: Obtén una única instancia de `SeleccionViewModel` con `viewModel()` en el Composable raíz y pásala a `EdificiosScreen` y a `HomeScreen` (en lugar de la lambda del ejercicio resuelto).

Implementación: En `MainScreen` se creó la instancia con `val seleccionViewModel: SeleccionViewModel = viewModel()`. Esta función entrega siempre la misma instancia mientras el `ViewModelStoreOwner` (la Activity) siga vivo, por lo que ambas

pantallas comparten el mismo objeto. Esa instancia se pasa como parámetro a `HomeScreen` y a `EdificiosScreen`. Para poder ejecutar y comprobar este paso, se mantiene temporalmente la lambda anterior y se agrega en `HomeScreen` una línea que muestra el valor del ViewModel, que por ahora aún dice «Ninguno».

Código agregado:

```
// Nuevo import
import androidx.lifecycle.viewmodel.compose.viewModel

// En MainScreen: única instancia del ViewModel compartido
val seleccionViewModel: SeleccionViewModel = viewModel()

// En el NavHost: se pasa el ViewModel a ambas pantallas
composable(Screen.Home.route) {
    HomeScreen(
        edificioSeleccionado = edificioSeleccionado,
        viewModel = seleccionViewModel
    )
}
composable(Screen.Edificios.route) {
    EdificiosScreen(
        viewModel = seleccionViewModel,
        onEdificioSeleccionado = { nombre -> edificioSeleccionado = nombre }
    )
}

// HomeScreen ahora recibe el ViewModel y muestra su valor
@Composable
fun HomeScreen(edificioSeleccionado: String, viewModel: SeleccionViewModel) {
    Column(/* ... */) {
        Text(text = "Bienvenido a la app de Edificios")
        Text(text = "Último edificio consultado: $edificioSeleccionado")
        Text(text = "Valor en ViewModel: ${viewModel.edificioSeleccionado}")
    }
}

// EdificiosScreen ahora recibe el ViewModel (junto a la lambda temporal)
@Composable
fun EdificiosScreen(
    viewModel: SeleccionViewModel,
    onEdificioSeleccionado: (String) -> Unit
) { /* ... */ }
```

Paso 3: Actualizar el ViewModel desde `EdificiosScreen`
Enunciado: En `EdificiosScreen`, al seleccionar un edificio, actualiza directamente el valor en el ViewModel en vez de invocar una función lambda.

Implementación: Se eliminó el parámetro lambda `onEdificioSeleccionado` de `EdificiosScreen` y también la variable de estado local de `MainScreen`. Ahora el botón «Ver» llama a `viewModel.seleccionarEdificio(nombre)`, que modifica la propiedad del ViewModel. `HomeScreen` todavía conserva la línea «Valor en ViewModel» del paso anterior, así que al pulsar «Ver» y volver a la pestaña Home se comprueba que el valor cambió. La línea «Último edificio consultado» ya no se actualizará, porque dependía de la lambda; se corrige en el siguiente paso.

Código modificado:

```
// En MainScreen: se elimina la variable local y la lambda
// var edificioSeleccionado by remember { mutableStateOf("Ninguno") }  <- eliminado

composable(Screen.Home.route) {
    HomeScreen(viewModel = seleccionViewModel)
}
composable(Screen.Edificios.route) {
    EdificiosScreen(viewModel = seleccionViewModel) // sin lambda
}

// EdificiosScreen: el botón actualiza directamente el ViewModel
@Composable
fun EdificiosScreen(viewModel: SeleccionViewModel) {
    // ...
    Button(onClick = { viewModel.seleccionarEdificio(nombre) }) {
        Text("Ver")
    }
}
```

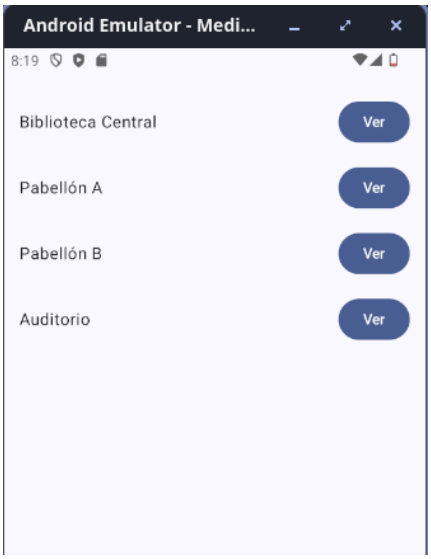


Figura 23: Paso 3 (a): en Edificios se pulsa «Ver» en un edificio, por ejemplo «Pabellón A».

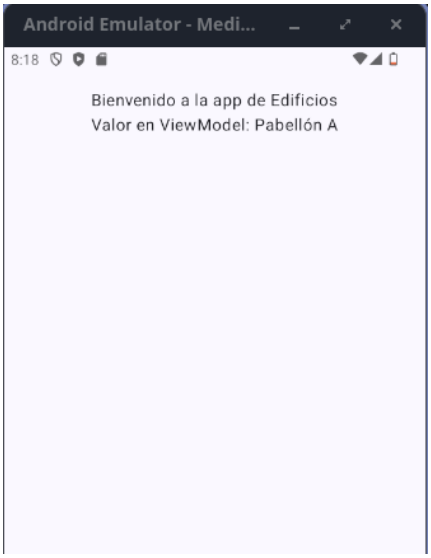


Figura 24: Paso 3 (b): en Home el valor del ViewModel cambió al edificio seleccionado.

Paso 4: Leer el valor del ViewModel en `HomeScreen`

Enunciado: En `HomeScreen`, lee el valor del ViewModel (con `collectAsState()` si usaste `StateFlow`) y muéstralo, verificando que el comportamiento visual sea el mismo que con la lambda.

Implementación: Como se usó `mutableStateOf` y no `StateFlow`, no hace falta `collectAsState()`: Compose observa directamente el estado y recompone `HomeScreen` cuando cambia. Se restauró el texto original «Último edificio consultado: …» leyendo `viewModel.edificioSeleccionado`, y se quitó la línea de comprobación del paso anterior. El resultado visual es idéntico al de la versión con lambda: se pulsa «Ver» en Edificios, se vuelve a Home y aparece el edificio elegido.

Código modificado:

```
@Composable
fun HomeScreen(viewModel: SeleccionViewModel) {
    // Se lee el estado del ViewModel (mutableStateOf no requiere collectAsState)
    val edificio = viewModel.edificioSeleccionado

    Column(/* ... */) {
        Text(text = "Bienvenido a la app de Edificios")
        Text(text = "Último edificio consultado: $edificio")
    }
}
```

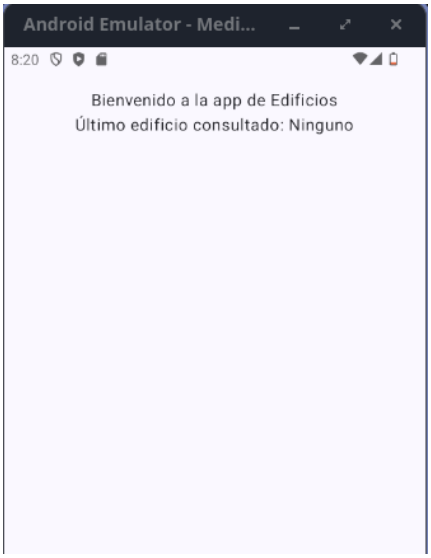


Figura 25: Paso 4 (a): Home antes de seleccionar, con «Último edificio consultado: Ninguno».

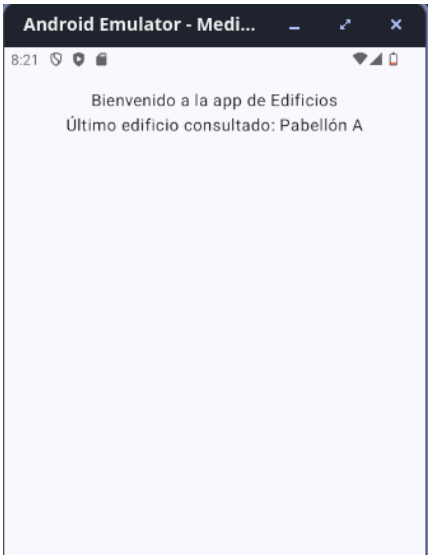


Figura 26: Paso 4 (b): Home después de pulsar «Ver» en un edificio, mostrando el valor del ViewModel.

Paso 5: Agregar la pestaña «Perfil»

Enunciado: Agrega una cuarta pestaña «Perfil» (u otra que prefieras) con un Composable simple de marcador de posición, y su `NavigationBarItem` correspondiente.

Implementación: Se agregó el objeto `Screen.Perfil` con la ruta `"perfil"` en la `sealed class`. Se incluyó en la lista `items`, se asignó el ícono `Icons.Default.Person` dentro del `when` (que debe ser exhaustivo, ya que `Screen` es sealed) y se registró un nuevo `composable(Screen.Perfil.route)` en el `NavHost`. Por último, se creó `PerfilScreen`, un Composable simple con un texto de marcador de posición. Con esto la barra inferior muestra cuatro pestañas y el ViewModel compartido sigue funcionando.

Código agregado:

```
// Nuevo import
import androidx.compose.material.icons.filled.Person

// En la sealed class
object Perfil : Screen("perfil", "Perfil")

// En MainScreen: se agrega a la lista de pestañas
val items = listOf(Screen.Home, Screen.Edificios, Screen.Mapa, Screen.Perfil)

// En el when de los íconos
Screen.Perfil -> Icons.Default.Person

// En el NavHost
composable(Screen.Perfil.route) {
    PerfilScreen()
}

// Nuevo Composable marcador de posición
@Composable
fun PerfilScreen() {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text(text = "Perfil de usuario (pendiente de implementar)")
    }
}
```

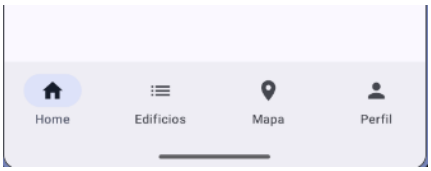


Figura 27: Paso 5 (a): barra de navegación con las cuatro pestañas.



Figura 28: Paso 5 (b): pestaña «Perfil» activa después de presionarla.

Entregables

El entregable solicitado es un informe con capturas de las cuatro pestañas funcionando, de la comunicación mediante el ViewModel compartido y una breve descripción de la implementación.

Capturas de las cuatro pestañas



Figura 29: Pestaña Home: muestra el último edificio consultado, leído desde el ViewModel compartido.



Figura 30: Pestaña Edificios: lista de edificios con su botón «Ver».



Figura 31: Pestaña Mapa: mapa estilizado del campus con la ubicación de los edificios.



Figura 32: Pestaña Perfil: Composable marcador de posición agregado como cuarta pestaña.

Comunicación mediante el ViewModel compartido

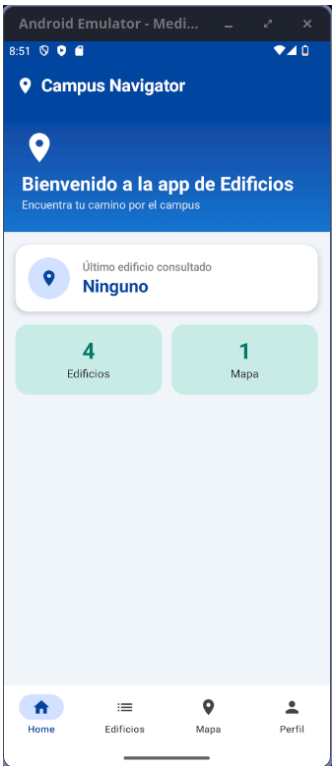


Figura 33: Home antes de seleccionar: «Último edificio consultado» muestra «Ninguno».

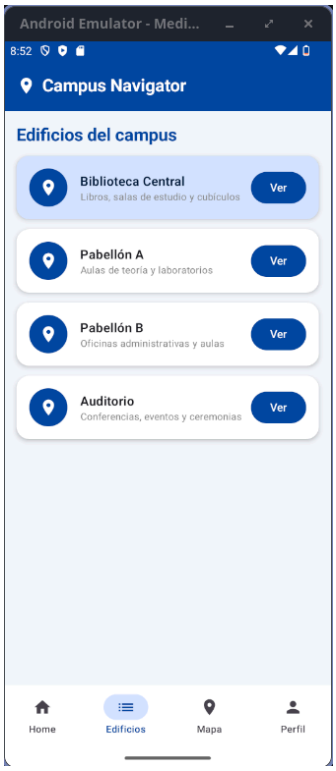


Figura 34: Edificios: se pulsa «Ver» en «Biblioteca Central»; la tarjeta queda resaltada porque lee el mismo ViewModel.

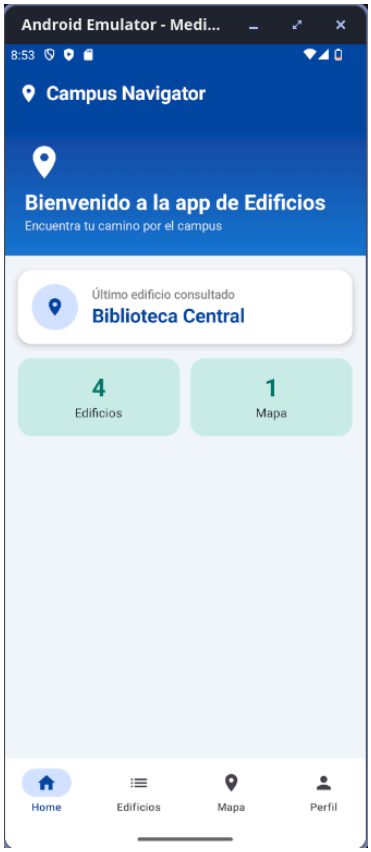


Figura 35: Home después de seleccionar: aparece «Biblioteca central», sin lambdas ni estado local.

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1: ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantallas Home, Edificios y Mapa navegables mediante NavigationBar, donde el edificio seleccionado en Edificios se refleja en Home)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Sí, se resolvió completamente. La `NavigationBar` inferior navega entre **Home**, **Edificios** y **Mapa** marcando siempre el ítem activo, y el edificio elegido en **Edificios** se refleja en **Home**, que pasa de mostrar «Ninguno» a mostrar el nombre seleccionado. Las capturas del emulador evidencian el recorrido completo de la solución.

Ejemplo: se pulsa **Ver** en el «Pabellón A» de la pestaña **Edificios** y, al volver a **Home**, el texto ya indica «Último edificio: Pabellón A».

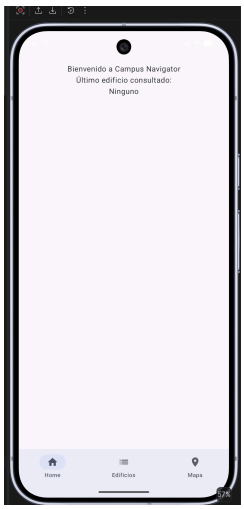


Figura 36: (a) Home inicial («Ninguno»)

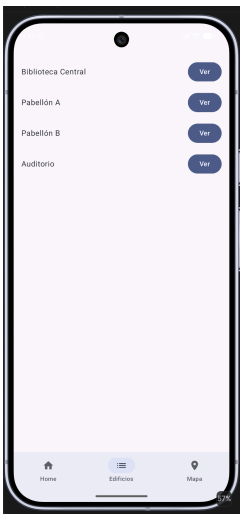


Figura 37: (b) Edificios (selección)

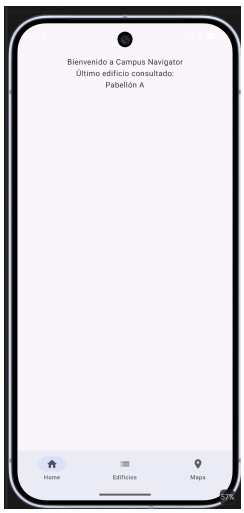


Figura 38: (c) Home ya actualizado



Figura 39: (d) Pantalla Mapa

Pregunta 2: En tu propia implementación, ¿qué diferencia encontraste entre comunicar Composables mediante una función lambda y hacerlo mediante un ViewModel compartido? ¿Cuál de las dos alternativas consideras una mejor práctica? Explica brevemente los motivos.

La lambda solo notifica el evento al Composable raíz, que guarda el estado en mutableStateOf; es simple, pero ese estado se pierde al recrear la Activity. El `ViewModel` compartido conserva el estado entre pantallas, rotaciones y cambios de configuración. Para este laboratorio basta la lambda, pero en una app real el `ViewModel` es mejor práctica: desacopla la UI y es más fácil de testear.

Ejemplo: el mismo edificio notificado de dos formas.

```
// Lambda: el estado vive en la UI y se pierde al recrear la pantalla
var edificio by remember { mutableStateOf("Ninguno") }
EdificiosScreen(onEdificioSeleccionado = { edificio = it })

```

```
// ViewModel compartido: el estado sobrevive a la navegación y a la rotación
class EdificioViewModel : ViewModel() {
    var edificio by mutableStateOf("Ninguno"); private set
    fun seleccionar(nombre: String) { edificio = nombre }
}
```

Pregunta 3: En forma individual, explica qué ventaja adicional ofrece un ViewModel frente a otras formas de mantener el estado al navegar entre pantallas. (Cada integrante del grupo debe incluir su propia respuesta en el informe).

Integrante 1

Frente a remember o a un objeto object global, el ViewModel mantiene el estado vivo durante toda la navegación y ante rotaciones o cambios de configuración, sin recrearse ni filtrar memoria. Además, al usar viewModelScope, las corrutinas se cancelan solas cuando la pantalla desaparece, lo que evita operaciones innecesarias y permite inyectar dependencias para las pruebas.

Ejemplo: si el usuario elige «Pabellón A» en Edificios y luego gira el teléfono, remember perdería el valor, pero el `ViewModel` seguiría mostrando «Pabellón A» en Home.

```
class EdificioViewModel : ViewModel() {
    var edificio by mutableStateOf("Ninguno"); private set
    fun seleccionar(nombre: String) {
        viewModelScope.launch { /* simular llamada a API o BD */ }
        edificio = nombre
    }
}
```

Integrante 2

La ventaja que noté al hacer el ejercicio es que el ViewModel separa el estado de la interfaz.

Al utilizar `remember`, el valor vive dentro de un Composable, y para compartirlo entre dos pantallas hay que subirlo al padre y pasarlo con lambdas. Pero al utilizar un ViewModel compartido, `EdificiosScreen` escribe y `HomeScreen` lee del mismo objeto.

Ejemplo: el usuario elige «Auditorio» en Edificios, rota el teléfono y cambia varias veces de pestaña. Home sigue mostrando «Auditorio» porque el dato está en el ViewModel y no en un Composable que se recrea.

```
class SeleccionViewModel(
    private val savedState: SavedStateHandle
) : ViewModel() {

    // StateFlow que se guarda y se restaura automáticamente
    val edificio: StateFlow<String> =
        savedState.getStateFlow("edificio", "Ninguno")

    fun seleccionar(nombre: String) {
        savedState["edificio"] = nombre
    }
}

// En HomeScreen:
// val edificio by viewModel.edificio.collectAsState()
```

III. CONCLUSIONES

- Se logró construir una aplicación con navegación entre pantallas utilizando `NavigationBar`, `NavHost` y una `sealed class Screen`, esto permite definir las rutas en un solo lugar y evita errores por escribir cadenas sueltas.
- Se comprobó que comunicar pantallas con una lambda funciona bien en ejemplos pequeños, pero obliga a subir el estado al Composable raíz y a pasar callbacks por cada pantalla. Con un `ViewModel` compartido, `EdificiosScreen` escribe y `HomeScreen` lee del mismo objeto, sin lambdas ni estado local en `MainScreen`. El comportamiento visual es el mismo, pero el código queda más ordenado y desacoplado.
- Se concluye que el `ViewModel` es la mejor opción para mantener el estado en una app real, porque sobrevive a rotaciones y cambios de configuración, y su ciclo de vida lo gestiona el sistema. También se vio que `mutableStateOf` y `StateFlow` sirven para exponer el estado: el primero es más directo y el segundo es más flexible y requiere `collectAsState()`. Por último, mejorar el diseño con Material 3 (barra superior, tarjetas y mapa estilizado) demostró que la interfaz puede cambiar sin tocar la lógica del estado.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.

[2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.

[3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.